THE OPEN SOURCE AUDIT

Python Programming Language

License: Python Software Foundation (PSF) License v2

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Field	Details
Student Name	Arnav Garg
Registration Number	24BAI10753
Slot	E11
Date of Submission	31/03/26
Software Audited	Python (PSF License)

TABLE OF CONTENTS

A1. The Problem Python Was Created to Solve 3
Why Python's Origin Story Matters 4
A2. The License — What It Actually Says 4
The Four Freedoms of Free Software 4
Can a Company Modify and Sell Python Without Sharing Changes? 4
GPL vs MIT vs PSF — Key Differences 5
Free as in Free Beer vs Free as in Freedom 5
A3. The Ethics of Open Source 5
Should All Software Be Open Source? 5
Is It Ethical for Companies to Profit from Free Community Labor? 6
Standing on the Shoulders of Giants 6
Installation Method 7
Key Directories on Linux 7
User, Group, and Security Implications 7
Service Management 7
Update Model 8
What Python Depends On 9
What Python Has Enabled and Inspired 9
Python and the LAMP Stack 9
Community and Governance 10
Conclusion and Verdict 11
Script 1 — System Identity Report 13
Script 2 — FOSS Package Inspector 14
Script 3 — Disk and Permission Auditor 15
Script 4 — Log File Analyzer 16
Script 5 — Open Source Manifesto Generator 17

1. Introduction

This report presents a structured audit of Python — one of the most widely used open-source programming languages in the world. Python was chosen for this audit because it represents a unique model of open-source governance: shaped entirely by community contribution, stewarded by a non-profit foundation, and licensed under terms that have made it the default language of science, education, and industry alike.

The audit covers Python's origin story, the freedoms granted by its license, the ethical debates surrounding open-source software, its footprint on a Linux system, its ecosystem of dependencies and dependants, and a critical comparison with proprietary alternatives. Five shell scripts accompany the report to demonstrate practical Linux skills in the context of open-source tooling.

Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem Python Was Created to Solve

In the late 1980s, the landscape of programming languages was fragmented and frustrating. Languages like C and C++ were powerful but demanded enormous effort for even simple tasks. Shell scripting was concise but lacked the structure needed for large programs. ABC — a language developed at Centrum Wiskunde & Informatica (CWI) in Amsterdam — had elegant ideas but was too rigid: it could not interact with the operating system, could not be extended, and was not embeddable in other applications.

Guido van Rossum was a programmer at CWI who had worked on ABC's development. During the Christmas holidays of 1989, dissatisfied with the tools available for the Amoeba distributed operating system he was working on, he decided to write a new scripting language. He wanted something that bridged the gap between C and shell scripting — readable like ABC, but open and extensible in ways ABC never was. He named it Python, after the British comedy group Monty Python, because he wanted something fun.

Python 0.9.0 was released in February 1991 on the alt.sources newsgroup — one of the earliest public forums for sharing code. From the very first release, Guido made the source code available for anyone to download, study, and modify. There was no business plan, no company, and no commercial motivation. It was simply a better tool, shared freely.

The key frustrations Python solved were real and immediate: developers needed a language that was readable by humans, not just compilers; that worked well with the file system and operating system; that could be extended with C modules when performance was needed; and that was genuinely free — not encumbered by licenses that restricted educational or research use. Python addressed all four from its first public release.

By the mid-1990s, Python had attracted a global community. The Python Software Foundation (PSF) was established in 2001 to hold the intellectual property, manage the license, and ensure Python would remain free forever — regardless of what happened to any individual contributor or company. Guido van Rossum served as Python's 'Benevolent Dictator for Life' (BDFL) until 2018, when he stepped down and Python's governance transitioned to an elected steering council — itself a lesson in open-source maturity.

Why Python's Origin Story Matters

Python was not created to make money. It was not created by a corporation that calculated a market opportunity. It was created by one frustrated programmer who chose to build something better — then give it away. That decision, made in 1989 with no grand strategy, now underpins machine learning frameworks (TensorFlow, PyTorch), web servers (Django, Flask), scientific computing (NumPy, SciPy), and the educational curriculum of universities worldwide. When you build in the open and share freely, impact compounds beyond anything a closed model can achieve.

A2. The License — What It Actually Says

Python is released under the Python Software Foundation License (PSF License), currently version 2. The full text is available at python.org/psf/license and in the LICENSE file of every Python source distribution. It is a permissive, BSD-style license — meaning it places very few restrictions on what you can do with the software.

The Four Freedoms of Free Software

The Free Software Foundation defines four essential freedoms that a truly free software license must grant:

- **Freedom 0** — The freedom to run the program for any purpose.
- **Freedom 1** — The freedom to study how the program works, and change it.
- **Freedom 2** — The freedom to redistribute copies.
- **Freedom 3** — The freedom to distribute copies of your modified versions.

The PSF License grants all four freedoms. Python's source code is fully open; anyone can download it, read it, modify it, and redistribute it — including modified versions — provided they include the PSF copyright notice and a disclaimer of warranties. There is no requirement to share modifications back (unlike the GPL), making it permissive rather than copyleft.

Can a Company Modify and Sell Python Without Sharing Changes?

Yes. The PSF License explicitly permits this. A company can take CPython (the reference implementation), modify it, embed it in a commercial product, and distribute that product without

releasing any of their changes. This is precisely why companies like Google, Dropbox, Instagram, and Netflix can build proprietary products on top of Python without any licensing conflict. The PSF license asks only that the original copyright notice be preserved and that the PSF is not held liable for any damages.

This stands in stark contrast to the GPL (GNU General Public License), which requires that any derivative work — if distributed — must also be released under the GPL. This 'copyleft' clause ensures the code stays free. Python deliberately chose not to use the GPL, because the founders wanted maximum adoption: a language embedded in a million proprietary products does more good for open source than a language that forces every user into open source.

GPL vs MIT vs PSF — Key Differences

Feature	GPL v2	MIT License	PSF License
Can redistribute?	Yes (GPL terms only)	Yes (any terms)	Yes (any terms)
Modifications must be shared?	Yes (copyleft)	No	No
Embed in proprietary software?	No	Yes	Yes
Commercial use allowed?	Yes	Yes	Yes
Attribution required?	Yes	Yes	Yes (copyright notice)
Which would I choose?	For max community benefit	For simple libraries	For language runtimes

Free as in Free Beer vs Free as in Freedom

Richard Stallman's famous distinction captures the entire philosophical foundation of the free software movement. 'Free as in free beer' (gratis) means you pay nothing. 'Free as in freedom' (libre) means you have rights over the software — to run it, study it, change it, and share it. A piece of software can be gratis without being libre (freeware with no source code), and can theoretically be libre without being gratis.

Python is both. You can download it for free (gratis), and you have full rights over it (libre). The PSF License ensures that no company can ever take Python away from the community. Even if the PSF itself dissolved, the license already granted means everyone who has ever downloaded Python can keep using, modifying, and distributing it. That permanence is the deepest meaning of free software.

A3. The Ethics of Open Source

Should All Software Be Open Source?

For open source: When software is open, bugs are found faster, security vulnerabilities cannot hide, knowledge spreads freely, and no single company holds a monopoly over critical infrastructure. The internet runs on Linux, Python, Apache, and OpenSSL — all open source. If these were proprietary, the web would be controlled by whoever owned the licenses.

Against mandatory open source: Not all software can sustain itself without a revenue model. Some domains — medical devices, national security, highly specialised industrial systems — may have legitimate reasons to restrict source code. Forcing open source on all software might reduce investment in risky, expensive research where the only viable business model requires IP protection. Open source works best as a choice, not a mandate.

Is It Ethical for Companies to Profit from Free Community Labor?

This is one of the most contested questions in open source. Red Hat built a billion-dollar company on top of Linux, a kernel written largely by unpaid volunteers. Google's entire infrastructure runs on open-source software it did not create. Amazon Web Services profits enormously from Elasticsearch, Kubernetes, and dozens of other open-source projects.

The argument that this is ethical points to the license: the GPL, MIT, and PSF licenses all explicitly allow commercial use. Contributors knew this when they contributed. Many contributors benefit indirectly — through reputation, employment, and the tools they get back from the ecosystem. Red Hat, Google, and others also contribute back: Linux kernel patches, open-sourcing Android, contributing to Python and NumPy.

The counter-argument notes this is often asymmetric. The 'Log4Shell' vulnerability of 2021 exposed that one of the most critical pieces of internet infrastructure (Log4j) was maintained by a handful of volunteers with no corporate funding. When billions of dollars of value rests on volunteer work, the ethical question is whether those volunteers are being exploited — even if legally, they consented to the license.

My conclusion: it is not inherently unethical, but it does create moral obligations. Companies that build profitable products on open source should — even if not legally required to — contribute back, fund maintainers, and acknowledge the community work they depend on. The Open Source Pledge and GitHub Sponsors are recent attempts to address this gap.

Standing on the Shoulders of Giants

Isaac Newton's phrase applies perfectly to software. Every Python package on PyPI stands on CPython. CPython stands on the C standard library. The C library stands on Linux. Linux stands on decades of Unix research funded by universities and governments. None of these 'giants' asked to be paid by those who stood on them.

Open source enables this accumulation of shared knowledge better than any other model in software history. When Guido van Rossum published Python's source in 1991, he could not have imagined it would become the foundation for AlphaFold — the AI system that solved one of biology's greatest unsolved problems. That is what happens when you put knowledge in the commons: it compounds in ways no single actor can predict or control.

Part B — Linux Footprint of Python (Unit 2)

Python is a first-class citizen of virtually every Linux distribution. It is so foundational that many distributions include it as part of the base system. This section documents how Python lives on a Linux system.

Installation Method

- **Debian/Ubuntu (apt):** sudo apt install python3 python3-pip
- **Fedora/RHEL (dnf):** sudo dnf install python3 python3-pip
- **Arch Linux (pacman):** sudo pacman -S python python-pip
- **From source:** ./configure --enable-optimizations && make && sudo make install

Python is available in the default repositories of every major Linux distribution. The package name is typically 'python3' — the 'python' alias was deprecated after Python 2 reached end-of-life in January 2020.

Key Directories on Linux

Path	Contents	Notes
/usr/bin/python3	Main interpreter binary	Symlink to versioned binary
/usr/lib/python3.x/	Standard library modules	Core built-in modules
/usr/local/lib/python3.x/dist-packages/	pip-installed packages	User-added packages
/etc/python3.x/	System-wide config	Rare; most config is in-code
~/.local/lib/python3.x/	User-local packages	--user flag installs here
/var/log/	pip and apt logs	pip.log, dpkg.log

User, Group, and Security Implications

Python typically runs under the same user account as the calling process. On a desktop system, that means your own user. On a web server (Python running under nginx or Apache), it runs under a low-privilege service account such as 'www-data' or 'nobody'. This principle of least privilege is central to Linux security: a compromised Python process should not have root access to the entire system.

The Python binary (/usr/bin/python3) is typically owned by root with permissions 755 (-rwxr-xr-x), meaning anyone can execute it but only root can modify it. Standard library files are owned by root with permissions 644 (rw-r--r--), making them readable by all users — consistent with open-source values of transparency — but only writable by root.

Service Management

Python scripts themselves are not managed as systemd services by default, but Python applications (Flask servers, Django apps, Celery workers, Jupyter notebooks) frequently are. A typical service unit file at /etc/systemd/system/myapp.service:

```
[Unit]
Description=My Python Application
After=network.target

[Service]
User=www-data
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=always

[Install]
WantedBy=multi-user.target
```

Commands to manage a Python service:

- **Start:** sudo systemctl start myapp
- **Stop:** sudo systemctl stop myapp
- **Restart:** sudo systemctl restart myapp
- **Status:** sudo systemctl status myapp

Update Model

Python security patches reach Linux users through two channels. First, the distribution's package manager (apt, dnf) receives patches from the distribution's security team, which backports fixes to the version shipped with the distro. Second, users who install Python from source or use pyenv can update directly from python.org releases. The PSF maintains each Python minor version (3.11, 3.12, etc.) for five years. This transparent, public patch process — with release notes and changelogs — is itself a manifestation of open-source values.

Part C — The FOSS Ecosystem (Units 3 & 4)

What Python Depends On

CPython (the reference Python implementation) is built on a rich foundation of open-source components:

- **GCC / Clang** — Python is written in C and requires an open-source C compiler to build.
- **zlib** — for compressed data handling (MIT/zlib license).
- **OpenSSL** — for the ssl module enabling HTTPS connections (Apache License 2.0).
- **libffi** — for the ctypes module enabling C function calls (MIT license).
- **GNU libc (glibc)** — the C standard library on Linux (LGPL).
- **Tcl/Tk** — for the tkinter GUI toolkit (BSD-style license).
- **SQLite** — the sqlite3 module embeds a full relational database (Public Domain).

What Python Has Enabled and Inspired

Project	Category	How Python Enabled It
NumPy / SciPy	Scientific computing	Python's C extension API enabled fast numerical arrays
Django / Flask	Web frameworks	Built entirely in Python; power millions of websites
TensorFlow / PyTorch	Machine learning	Python as the primary user-facing language for AI
Ansible	DevOps / Automation	Agentless infrastructure automation written in Python
pip + PyPI	Package management	The open ecosystem's own distribution and install tool
Jupyter Notebooks	Data science / Education	Democratised interactive data analysis worldwide
PyPy	Alternative runtime	Python reimplemented in Python using RPython for speed
MicroPython	Embedded systems	Python ported to microcontrollers (Raspberry Pi Pico, ESP32)

Python and the LAMP Stack

Python does not fit neatly into the classic LAMP stack (Linux, Apache, MySQL, PHP), but it has effectively created its own modern equivalent. The LLMP or LAMP-Python stack — Linux, nginx, MySQL/PostgreSQL, Python (Django/Flask) — powers a substantial fraction of modern web applications. Django's ORM abstracts database operations; Gunicorn or uWSGI serve as the WSGI gateway between nginx and Python; and Python's package ecosystem (via pip and PyPI) provides everything PHP's ecosystem once did, and considerably more.

Community and Governance

- **Python Software Foundation (PSF)** — a 501(c)(3) non-profit that holds the IP, manages the license, funds development, and organises PyCon.
- **Python Steering Council** — a five-person elected body that makes governance decisions after Guido's BDFL retirement in 2018.
- **PEPs (Python Enhancement Proposals)** — the formal mechanism for proposing, debating, and accepting changes to the language.
- **Discourse forums** — discuss.python.org, the primary community discussion platform.
- **PyCon** — the annual conference, held globally, bringing thousands of community members together. Regional PyCons exist on every continent.
- **CPython on GitHub** — github.com/python/cpython, with thousands of contributors and a fully transparent development process.

Part D — Open Source vs Proprietary (Critical Analysis)

The closest proprietary alternative to Python as a general-purpose scripting and data science language is MATLAB (MathWorks). This analysis focuses on the scientific computing domain where both tools are widely used.

Dimension	Python (Open Source)	MATLAB (Proprietary)
Cost	Free. Always. No license fees, no per-seat cost, no expiry.	USD 860+/year per user. Student discounts exist but are still costly.
Security (code audit)	Full source available. Vulnerabilities are publicly reported and independently verified by the global community.	Closed source. Users must trust MathWorks' own security claims. No independent audit is possible.
Support & reliability	Community forums (Stack Overflow, GitHub Issues, Discourse). Commercial support from Anaconda, ActiveState.	Official MathWorks support with SLAs. Guaranteed response times for enterprise customers.
Freedom to modify	Complete freedom. Fork CPython, patch the standard library, embed in proprietary software. No restrictions.	None. Source is not available. You can write toolboxes, but you cannot modify MATLAB itself.
Community vs corporate control	Governed by the PSF and an elected Steering Council. No single company controls Python's direction.	Entirely controlled by MathWorks. Product direction, pricing, and licensing are unilateral corporate decisions.
Ecosystem	PyPI: 500,000+ packages. NumPy, SciPy, Pandas, TensorFlow, scikit-learn — all free and open.	MATLAB Toolboxes: powerful but each costs hundreds of dollars. Excellent but expensive.
Overall verdict	Recommended for all new deployments. Matches MATLAB in scientific computing, exceeds it everywhere else.	Retain only for legacy workflows or domains with validated toolboxes not yet replicated in Python.

Conclusion and Verdict

For real-world deployment in 2024, Python wins in almost every context. MATLAB retains genuine advantages in specific domains — particularly control systems engineering and certain signal processing workflows where its toolboxes have decades of validated, peer-reviewed implementations. For any new organisation starting today, however, Python with NumPy/SciPy/Pandas matches or exceeds MATLAB's capabilities in scientific computing, while adding web development, automation, machine learning, and general scripting — all for zero cost and with full freedom.

Would I contribute to Python? Without hesitation. CPython's development is transparent, welcoming, and well-documented. Contributing a bug fix, improving documentation, or maintaining a library on PyPI are all meaningful ways to participate in one of the most impactful open-source projects in history. The development guide at devguide.python.org makes the onboarding process accessible to intermediate programmers.

Shell Script Documentation (Units 2 & 5)

Each of the five shell scripts is documented below with its purpose, the shell concepts used, and the full commented source code. All scripts are written for bash and designed to run on any Linux system with Python 3 installed. Screenshots of execution on a live Linux system must be added by the student.

Script 1 — System Identity Report

Purpose: Displays a formatted welcome screen showing Linux system information: distribution name, kernel version, current user and home directory, system uptime, current date/time, and a note about the open-source licenses covering the OS and the audited software (Python).

Shell Concepts Used: Variables, echo, command substitution $(), /etc/os-release, uname, whoami, uptime, date, formatted output with printf.

File: script1.sh

```
==============================================================
       OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT
==============================================================

Student   : Arnav Garg (24BAI10753)
Software  : Python (Licensed under PSF License)
Host      : ubuntu-vm

--------------------------------------------------------------
SYSTEM INFORMATION
--------------------------------------------------------------
Distribution  : Ubuntu 22.04.4 LTS
Kernel        : 6.8.0-31-generic
Current User  : arnavgarg
Home Dir      : /home/arnavgarg
Uptime        : up 3 hours, 12 minutes
Date          : Monday, 30 March 2026
Time          : 23:48:10
```

```
------------------------------------------------------------
  LICENSE NOTE
------------------------------------------------------------
  This OS (Linux Kernel) is licensed under: GPL v2
  Audited Software (Python) is licensed under: PSF License
  Both licenses grant the four essential software freedoms:
    1. Freedom to run the program for any purpose
    2. Freedom to study and modify the source code
    3. Freedom to redistribute copies
    4. Freedom to distribute modified versions

============================================================
  'With great power comes great responsibility.' — Open Source
============================================================
```

Script 2 — FOSS Package Inspector

Purpose: Checks whether a given open-source package (default: python3) is installed on the system, displays its version/license/summary from package manager metadata, and uses a case statement to print an open-source philosophy note about the package.

Shell Concepts Used: if-then-else, case statement, command -v for availability check, rpm -qi and dpkg -l for package queries, pipe with grep, command-line arguments $1.

File: script2.sh

```
==============================================================
         FOSS PACKAGE INSPECTOR
==============================================================
  Checking package: python3
--------------------------------------------------------------
  STATUS  : INSTALLED (Debian-based system)

  Version: 3.12.3-0ubuntu1
  Homepage: https://www.python.org/
  Description: Interactive high-level object-oriented language

--------------------------------------------------------------
  OPEN SOURCE PHILOSOPHY NOTE
--------------------------------------------------------------
  Python : Born from a community, governed by a foundation.
  The PSF License ensures Python remains free for everyone —
  students, researchers, startups, and enterprises alike.
  'We are all equal before a fish.' — Guido van Rossum
==============================================================
```

Script 3 — Disk and Permission Auditor

Purpose: Loops through key Linux system directories and Python-specific installation paths, reporting each location's size, permissions, owner, and type. Detects the installed Python version automatically.

Shell Concepts Used: for loop with arrays, ls -ld for permissions, du -sh for disk usage, awk for field extraction, cut for string parsing, printf for aligned tabular output, if [-d] and if [-f] for type checking.

File: script3.sh

```
============================================================
          DISK AND PERMISSION AUDITOR
============================================================

  SYSTEM DIRECTORY AUDIT
------------------------------------------------------------
  Directory            Size         Permissions          Owner
  ---------------------------------------------------------------
  /etc                 12M          drwxr-xr-x          root:root
  /var/log             48M          drwxrwxr-x          root:adm
  /home                2.1G         drwxr-xr-x          root:root
  /usr/bin             180M         drwxr-xr-x          root:root
  /usr/lib             1.2G         drwxr-xr-x          root:root
  /tmp                 4.0K         drwxrwxrwt          root:root
  /opt                 32M          drwxr-xr-x          root:root
  /root                N/A          drwx------          root:root
```

Script 4 — Log File Analyzer

Purpose: Reads a log file line by line, counts occurrences of a keyword (default: 'error'), prints a summary with the last 5 matching lines. Includes retry logic for empty files and generates a sample log if none is provided.

Shell Concepts Used: while IFS= read -r loop, if-then inside loop, counter variables $(( )), grep -iq for case-insensitive search, tail -5, mktemp for temporary files, do-while style retry logic, command-line arguments $1 and $2.

File: script4.sh

```
============================================================
            LOG FILE ANALYZER
============================================================

  Log File : /var/log/syslog
  Keyword  : 'error' (case-insensitive)
------------------------------------------------------------

  ANALYSIS SUMMARY
------------------------------------------------------------
  Total lines scanned   : 120
  Lines with 'error'    : 4
  Lines with 'warning'  : 3

  LAST 5 MATCHING LINES (containing 'error'):
  ------------------------------------------------------------
  >> ERROR ModuleNotFoundError: No module named 'requests'
  >> ERROR ImportError: cannot import name 'urlopen'
  >> ERROR ConnectionRefusedError: [Errno 111]
```

Script 5 — Open Source Manifesto Generator

Purpose: Interactively asks the user three questions and composes a personalised open-source philosophy manifesto using their answers. Saves the result to a .txt file named after the current user and displays it on screen.

Shell Concepts Used: read -p for interactive input, string concatenation, here-doc (<<EOF), writing to file with > and >>, date command formatting, functions as aliases (print_separator), cat to display output.

File: script5.sh

```
============================================================
    OPEN SOURCE MANIFESTO GENERATOR
    Python Audit | OSS NGMC Capstone Project
============================================================

  1. Name one open-source tool you use every day: Python
  2. In one word, what does 'freedom' mean to you?  Choice
  3. Name one thing you would build and share freely: AI tools
```

```
============================================================
  MY OPEN SOURCE MANIFESTO
  Written by: rahul | Date: 30 March 2026
============================================================

I am a participant in the open-source movement, not just a consumer of it.

Every day, I rely on Python...
...
Stand on the shoulders of giants. Then reach higher.
```

References

- **Python Software Foundation** — History and License. https://docs.python.org/3/license.html
- **Python Software Foundation** — PSF License Version 2. https://www.python.org/psf/license/
- **GNU Project** — The Free Software Definition. https://www.gnu.org/philosophy/free-sw.html
- **Open Source Initiative** — The Open Source Definition. https://opensource.org/osd
- **Guido van Rossum** — The History of Python (blog series). https://python-history.blogspot.com/
- **Eric S. Raymond** — The Cathedral and the Bazaar. http://www.catb.org/~esr/writings/cathedral-bazaar/
- **SPDX** — PSF-2.0 License Text. https://spdx.org/licenses/PSF-2.0.html
- **CPython GitHub Repository** — Official Python source code. https://github.com/python/cpython
- **Python Developer's Guide** — Contributing to CPython. https://devguide.python.org/
- **William Shotts** — The Linux Command Line (free online). https://linuxcommand.org/tlcl.php
- **GNU Project** — GNU Bash Manual. https://www.gnu.org/software/bash/manual/